Cloud-Edge Collaborative Inference with Large and Small Language Models: A Survey*

*Note: Sub-titles are not captured in Xplore and should not be used

Jiang Zhaobai
School of International
Beijing University of Posts and Telecommunications (BUPT)
Beijing, China
1783148122@qq.com

***Abstract*—The proliferation of Large Language Models (LLMs) has revolutionized natural language processing, yet their substantial computational and memory demands hinder deployment on resource-constrained edge devices. Simultaneously, Small Language Models (SLMs) offer efficiency but may lack the advanced capabilities of their larger counterparts. This survey explores the burgeoning field of cloud-edge collaborative inference, a paradigm that synergizes the strengths of cloud-based LLMs and edge-deployed SLMs. We systematically review fundamental concepts, system architectures, and the inherent challenges of such collaboration. Key strategies are examined, including dynamic task allocation, speculative decoding, multi-model collaborative architectures, and lightweight model adaptation techniques. Furthermore, the survey discusses benchmarking practices, real-world applications, and prominent open challenges, such as system scalability, privacy preservation, and energy efficiency. This work aims to provide a comprehensive overview for researchers and practitioners, highlighting current advancements and future directions in enabling intelligent, efficient, and secure LLM services across the cloud-edge continuum.**

***Index Terms*—Cloud-Edge Computing, Large Language Models (LLMs), Small Language Models (SLMs), Collaborative Inference, Model Optimization, Survey.**

I. INTRODUCTION

Large Language Models (LLMs) like GPT-3 [1], PaLM [2], and LLaMA [3] have demonstrated remarkable capabilities in understanding and generating human-like text, transforming various applications from automated customer support to content creation. However, these models typically comprise billions of parameters, necessitating significant computational resources, extensive memory, and high energy consumption for both training and inference. These requirements make their direct deployment on resource-constrained edge devices (e.g., smartphones, IoT devices, autonomous vehicles) challenging, if not infeasible.

Conversely, edge computing offers benefits such as low latency, reduced bandwidth usage, and enhanced data privacy by processing data closer to the source [4]. To leverage these advantages, Small Language Models (SLMs) have been developed. SLMs are designed to be lightweight and efficient, making them suitable for on-device execution. While SLMs can handle simpler tasks effectively, they often cannot match the sophisticated reasoning and generation capabilities of LLMs.

This disparity has spurred research into cloud-edge collaborative inference, a paradigm that aims to combine the best of both worlds: the power of cloud-based LLMs and the efficiency and responsiveness of edge-deployed SLMs [5], [6]. Such collaboration enables complex AI tasks to be intelligently partitioned and executed across the cloud-edge continuum, optimizing for performance, latency, cost, and privacy. Recent surveys have touched upon aspects like AI-Generated Content (AIGC) in mobile edge networks [7] or on-device LLMs [8], but a holistic examination of the collaborative inference mechanisms between large and small models in a cloud-edge setting is still emerging.

This survey addresses this gap by providing a comprehensive review of cloud-edge collaborative inference strategies specifically tailored for LLMs and SLMs. We delve into the foundational concepts, system architectures, and the inherent challenges. We then explore various techniques, including dynamic task allocation, speculative decoding frameworks, multi-model cooperation, and lightweight adaptation methods. Furthermore, we discuss benchmarks, deployment considerations, and critical open research challenges. The goal of this survey is to offer a structured understanding of this rapidly evolving field and to identify promising avenues for future research and development.

The remainder of this paper is organized as follows: Section II discusses the fundamentals of cloud-edge systems and language models. Section III explores core collaborative inference and task allocation strategies. Section IV delves into model optimization and lightweight deployment techniques. Section V covers benchmarks, applications, and open challenges. Finally, Section VI concludes the survey.

II. BACKGROUND AND FUNDAMENTALS

This section lays the groundwork by discussing the architectures of edge-cloud computing, the characteristics of edge devices, and the trends in language model development that necessitate collaborative approaches.

A. Edge-Cloud Computing Architecture

The edge-cloud collaborative inference framework strategically distributes tasks between powerful cloud servers and resource-aware edge devices. The general architecture, as conceptualized in Figure 1, involves user requests originating at an edge device, which may host an SLM. Based on the request's complexity, system load, and desired Quality of Service (QoS), tasks can be processed locally, offloaded entirely to a cloud-hosted LLM, or handled collaboratively.

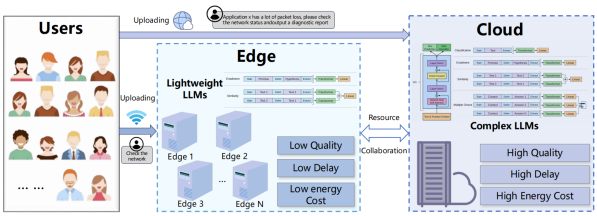

Fig. 1. Overall Architecture of Cloud-Edge Collaborative LLM Services, illustrating user interaction with edge and cloud layers for LLM inference. (Inspired by PerLLM, Figure 1)

Several approaches aim to balance model performance, communication costs, and real-time responsiveness. For instance, HybridSD [9] focuses on visual semantic planning where the cloud LLM handles early denoising stages, and the edge SLM refines details. CE-COLLM [10] introduces confidence-based dynamic inference allocation, processing high-confidence tokens on the edge and offloading low-confidence ones to the cloud. This dynamic partitioning is crucial for optimizing resource utilization and latency.

B. Characteristics and Constraints of Edge Devices

Edge devices, such as smartphones, wearables, and embedded systems in vehicles, are characterized by [11]:

1) Limited Computational Power: CPUs and GPUs on edge devices are less powerful than those in cloud data centers.
2) Restricted Memory and Storage: Available RAM and storage are often orders of magnitude smaller.
3) Energy Constraints: Battery life is a critical concern, especially for mobile devices.
4) Heterogeneity: A wide variety of hardware platforms and software environments exist, posing challenges for standardized deployment.

These constraints significantly impact the feasibility of deploying large AI models directly on-device and necessitate techniques like model compression, quantization, or collaborative approaches.

C. Trends in Large and Small Language Models

The field of language modeling is witnessing a dual trend. On one hand, LLMs like GPT-4 continue to scale up in parameters and capabilities, trained on vast text corpora to achieve profound linguistic understanding [1]. On the other hand, there is a growing interest in developing SLMs (e.g., TinyLlama [12], Phi-2 [13]) that maintain competitive performance while being significantly smaller and more efficient. Techniques such as knowledge distillation [14], quantization [15], and pruning are instrumental in creating these edge-friendly SLMs. This divergence motivates collaborative systems that can intelligently leverage the strengths of both large cloud models and lightweight edge models.

TABLE I
KEY CHARACTERISTICS: EDGE SLMS VS. CLOUD-EDGE COLLABORATION.

Feature Dimension	Edge SLM Only	Cloud-Edge Collab.
Model Capability	Limited reasoning; Narrow scope; Simple tasks	LLM power + SLM flexibility; Handles diverse tasks
Resource Demand	Low (for constrained devices)	Optimized allocation; Cloud on demand
Inference Latency	Very low (local exec.)	Reduced perceived latency (edge-first, collab. mechanisms)
Network Dependency	Offline/poor network capable	Reduced dependency on high-quality network
Data Privacy	Local processing, better privacy	Sensitive data local; Balances privacy & function
Deployment Cost	Relatively low edge cost	Optimized cost-effectiveness
Personalization	Easy on-device fine-tuning	Cloud general knowledge + Edge personalized data
Key Challenges	Limited capability	System complexity, collab. overhead, scheduling, security

III. CORE COLLABORATIVE STRATEGIES

Efficient collaboration between cloud LLMs and edge SLMs hinges on intelligent task allocation, optimized inference pathways, and effective communication. This section reviews key strategies.

A. Resource- and Uncertainty-Aware Task Allocation

Dynamically adjusting the inference path based on request characteristics and system states is crucial. PerLLM [16] introduces a personalized scheduling framework using an upper-confidence bound strategy to optimize service deployment and resource allocation across edge and cloud. EdgeLLM [17] proposes a value-density-first (VDF) scheduling algorithm, prioritizing tasks by their real-time cost-effectiveness. Task structure also offers optimization opportunities. Division-of-Thoughts (DoT) [18] decomposes complex queries into parallelizable subtasks, enabling model granularity adaptation. Approaches like FS-GEN [19] categorize SLMs as System

1 (fast, intuitive) and LLMs as System 2 (slow, deliberative), triggering LLM intervention based on SLM uncertainty.

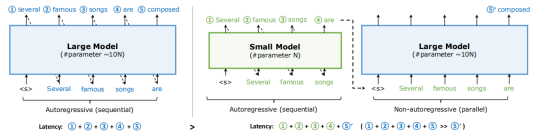

Fig. 2. Decision Logic for Request Handling in the VELO Framework. The edge server selects between returning a cached answer, augmenting the request before cloud LLM invocation, or directly querying the cloud LLM. (Inspired by VELO, Fig. 1)

B. Speculative Decoding and Collaborative Verification

Conventional autoregressive LLM inference is token-by-token, leading to high latency. Speculative decoding [20] addresses this by using a smaller, faster "draft" model (often an SLM on the edge) to generate a sequence of candidate tokens. These drafts are then verified in parallel by a larger, more powerful "target" model (the LLM in the cloud). If accepted, multiple tokens are produced in a single step, accelerating inference.

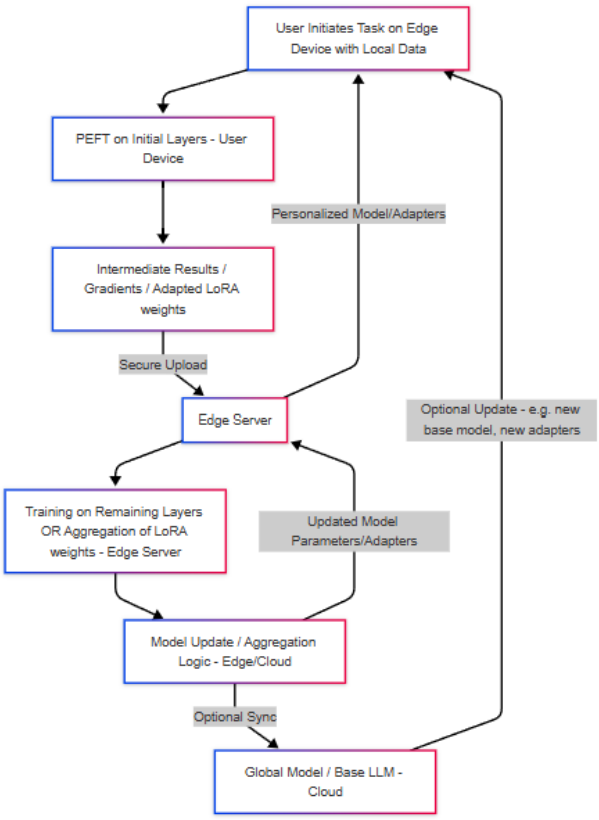

Fig. 3. Comparison of Standard Autoregressive Decoding and BiLD's Collaborative Speculative Decoding. BiLD utilizes a small model for initial token generation and a large model for verification and refinement, improving latency. (Inspired by BiLD, Figure 1)

1) Edge Draft and Cloud Validation: BiLD [21] (from '010Speculative Decoding...') integrates a small autoregressive model for drafting and a larger model for assistance and verification. RSD [22] and SpecExec [23] enhance this by incorporating controlled bias strategies. SMART [24] uses score-based evaluations to identify uncertain reasoning steps in the SLM and injects LLM-generated reasoning only when necessary. DiSCo [25] and AdaServe [26] focus on optimizing Quality of Experience (QoE) metrics like Time-To-First-Token (TTFT) and Token-By-Token time (TBT) through adaptive scheduling and token acceptance strategies.

2) Self-Speculative Decoding: To avoid the overhead of a separate draft model, self-speculative decoding reuses parts of the target LLM itself. Kangaroo [27] uses shallow subnetworks of the target LLM as a self-draft model. SWIFT [28] exploits inherent layer sparsity in LLMs, dynamically selecting skip-layer configurations to create draft pathways.

3) Semantic and Skeleton Completion: This paradigm involves the edge SLM generating a coarse-grained draft or "skeleton" of the response. The cloud LLM then refines or "elaborates" on this skeleton, correcting errors and adding details [29]. PICE [30] proposes a progressive scheme where the LLM generates a task sketch on the cloud, and the SLM fills in details on the edge.

4) Token-Level Tree Validation: Instead of linear draft verification, token trees allow branching into multiple candidate paths. LLMCad [31] uses a tree-based generation and validation framework with a confidence-based branching scheduler. OPT-Tree [32] dynamically adjusts tree construction to maximize accepted token length.

C. Historical Enhancement and Retrieval-Augmented Collaboration

Leveraging historical user interactions with cloud LLMs can enhance on-device inference. Ding et al. [33] propose storing past question-answer pairs on mobile devices to improve the on-device SLM's reasoning via nearest-neighbor retrieval. VELO [6] (from '003VELO.pdf') extends this to edge-side scheduling by caching embeddings of past LLM requests/responses in a vector database on edge servers, as illustrated in its Fig. 1. HybridRAG [34] integrates cloud-based retrieval models and LLMs with lightweight on-device predictors, where the cloud retrieves and compresses documents into "memory units" for the edge SLM to refine its predictions.

IV. OPTIMIZATION AND DEPLOYMENT CONSIDERATIONS

Effectively deploying collaborative LLM systems requires optimizing models for edge constraints and ensuring efficient knowledge transfer between cloud and edge.

Fig. 4. General Workflow for PEFT-based Collaborative LLM Fine-tuning/Training in a Cloud-Edge Environment. Users perform PEFT on initial layers on-device, while edge servers handle more complex later layers or aggregate adapter updates, with potential cloud synchronization. (Inspired by '008ResourceAllocation...pdf', Section 3 System Model)

TABLE II
TECHNICAL DETAILS OF REPRESENTATIVE COLLABORATIVE INFERENCE STRATEGIES.

Strategy Category	Key Mechanism / Algorithm Focus	Edge Model Resp.	Cloud Model Resp.	Comm. Content
Task Allocation (Uncertainty-Aware, Resource-Aware)	Value-density scheduling (EdgeLLM), UCB-based optimization (PerLLM), System 1/2 cognitive model (FS-GEN), MARL for QoS (VELO)	Initial processing, Low-uncertainty tasks, Req. feature extraction (VELO)	Complex/High-uncertainty tasks, Oversight, Global policy training (VELO)	Task desc., Partia data/results, Vecto query outcome (VELO)
Speculative Decoding (Draft & Verify / BiLD)	SLM drafts tokens, LLM verifies in batch, Adaptive token acceptance, Fall-back/Rollback policies (BiLD)	Generate draft token sequences (fast)	Verify drafts, Generate if drafts fail, Correct/Refine based on poli-cies	Draft tokens, Verifica tion signals, Contro signals
Skeleton Completion / Progressive Inference	SLM generates structural outline, LLM elaborates details (PICE)	Generate "skeleton" or coarse response, Fill in details based on cloud sketch	Generate task sketch, Refine, Add details, Ensure coherence	Skeleton, Task sketch Refined sections, Ed its
Retrieval Augmented (Historical/External Data)	Utilize cached/retrieved context to augment local SLM input (VELO, HybridRAG)	Query local/edge vector DB, SLM inference with augmented context	Manage/Update central knowledge/cache, Retrieve & compress context for edge	Query vectors, Com pressed context, QA pairs, Embeddings
Guidance-based Knowledge Transfer (GKT-like)	LLM (Teacher) generates compact guidance prompts for SLM (Student) (GKT)	Execute inference based on guidance prompts from cloud	Generate task-specific guidance prompts (batch generation)	Compact guidanc prompts, Use settings

A. Lightweight Deployment and Knowledge Transfer

1) Parameter-Efficient Fine-Tuning (PEFT): Techniques like LoRA (Low-Rank Adaptation) [35] and BitFit [36] adapt pre-trained LLMs to specific tasks or user data by fine-tuning only a small subset of parameters. This is crucial for personalization on the edge with minimal overhead, as detailed in Figure 4. Liu et al. (`008ResourceAllocation...` [11]) proposed an end-to-end training loop integrating edge proxies with lightweight adapters. The work in `008ResourceAllocation for Stable LLM Training in Mobile Edge Computing.pdf` also leverages PEFT, particularly in its system model (Section 3, Figure 1 of that paper) where users fine-tune initial layers and edge servers handle later ones.

2) Adapter-Based Modular Designs: Adapters are small neural network modules inserted into pre-trained models. They can be trained for specific tasks or domains and selectively activated at inference time, enabling flexible model specialization [38].

3) Cross-Domain Knowledge Transfer and Distillation: Knowledge distillation allows transferring capabilities from a large "teacher" LLM (in the cloud) to a smaller "student" SLM (on the edge) [14]. GKT [37] (from `2GKT.pdf`) exemplifies this with its teacher-student architecture, as shown in Figure 5. DC-CCL [39] employs vertical model partitioning with a lightweight proxy model on the edge trained via distillation. FedMKT [40] explores bidirectional knowledge transfer.

B. Quantization and Pruning in Edge Deployments

To further reduce model size and computational cost for edge deployment, quantization and pruning are widely used:

- **Quantization:** Reduces the precision of model weights and activations (e.g., from 32-bit floating point to 8-bit integer) [15]. EdgeLLM [17] incorporates a hardware scheduling module that dynamically adjusts quantization bit width. EdgeMoE (from `1EdgeMoE.pdf`) heavily utilizes expert-wise bitwidth adaptation (its Section 3.2), conceptually shown in Figure 6.

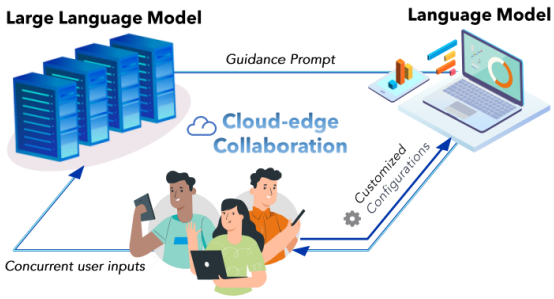

Fig. 5. The GKT Framework for Cloud-Edge Collaboration. A cloud-based teacher LLM generates guidance prompts for an edge-based student SLM, enabling efficient knowledge transfer and customized responses. (Inspired by GKT, Figure 1)

- **Pruning:** Removes less important weights or structural components (e.g., attention heads, layers) from the model.

These techniques are often applied to SLMs before deployment or can be part of the adaptation process from an LLM.

C. Communication Optimization

Efficient communication is vital. Strategies include:

- Specialized models (like a smaller GPT variant) to decompose user requests into executable task sequences, minimizing unnecessary data transfer [29].
- Entropy models for feature compression when intermediate data must be transmitted [41].
- EdgeShard uses dynamic token-level collaboration. GKT [37] (from `2GKT.pdf`) also emphasizes reducing data transmission costs by sending brief guidance prompts.

V. BENCHMARKS, APPLICATIONS, AND OPEN CHALLENGES

This section discusses how collaborative LLM systems are evaluated, their applications in vertical domains, and the key challenges that remain.

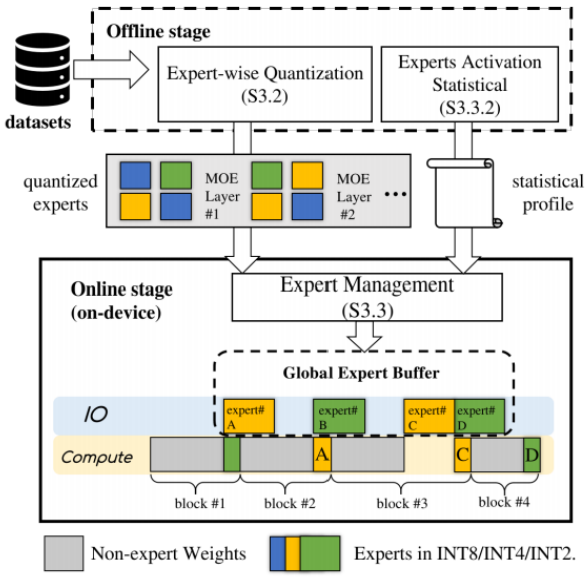

Fig. 6. System Architecture and Workflow of EdgeMoE. This illustrates how EdgeMoE manages expert models on-device through techniques like expert-wise bitwidth adaptation and in-memory expert management. (Inspired by EdgeMoE, Figure 5)

A. Benchmarks, Datasets, and Evaluation Protocols

Evaluating cloud-edge collaborative LLM systems requires considering multiple dimensions:

1) Latency: End-to-end response time, TTFT, TBT.
2) Accuracy/Quality: Standard NLP metrics.
3) Cost: Communication overhead, computational cost, energy consumption.
4) Privacy: Degree of raw data exposure.

The papers studied (e.g., VELO, PerLLM, GKT, EdgeMoE) define their own experimental setups and baselines, using datasets like GSM8K, CSQA, XSUM, CNN/DailyMail, SAMsum, or oasst1. For example, PerLLM ([16], Section 4.1) details its evaluation setup and models.

B. Applications in Vertical Domains

Cloud-edge collaborative LLMs are finding applications in various fields:

- **Autonomous Driving:** ADAS [43], EC-Drive [44].
- **Mobile Assistants and IoT:** GKT [37], EdgeMoE [34].
- **Healthcare:** Personalized monitoring.

C. Open Challenges and Future Directions

Despite significant progress, several challenges and opportunities remain:

1) System Scalability and Robustness: AdaptLink [45], EdgeMoE [34].
2) Adaptive and Dynamic Scheduling: PerLLM [16], VELO [6].
3) Privacy-Preserving Collaboration: CoGenesis [30], SuperICL [46].
4) Energy-Efficient Design (Green AI): PerLLM [16], `008ResourceAllocation...` [11].
5) Multimodal Integration.
6) Continual Learning and Personalization.
7) Trustworthy and Secure AI.

VI. CONCLUSION

Cloud-edge collaborative inference for Large and Small Language Models represents a promising direction to harness the power of modern AI while respecting the constraints of edge computing. This survey has provided a comprehensive overview of this dynamic field, covering fundamental architectures (illustrated by PerLLM [16], VELO [6]), core collaborative strategies such as task allocation (PerLLM, VELO) and speculative decoding (BiLD [21], Fast Inference [20]), optimization techniques for model deployment (GKT [37], EdgeMoE [34]) and knowledge transfer, and practical considerations like benchmarking and applications. Insights from these key works highlight the diverse approaches to achieving efficient and effective cloud-edge synergy.

The key finding of this survey is that significant advancements have been made in designing sophisticated mechanisms that allow LLMs and SLMs to work in synergy, leading to improvements in latency, efficiency, and personalization. However, the field is still evolving. Addressing the open challenges related to scalability, privacy, energy efficiency, and robust multimodal collaboration will be critical for the widespread adoption of these intelligent systems. Future research should focus on developing more adaptive, secure, and resource-aware frameworks that can unlock the full potential of LLMs across the diverse landscape of cloud and edge devices, ultimately enabling more pervasive and responsive AI-powered services.

ACKNOWLEDGMENT

The author would like to thank the School of International at Beijing University of Posts and Telecommunications, the instructors and reviewers for their guidance, and the numerous researchers whose publications formed the basis of this survey and have significantly contributed to the advancements in cloud-edge collaborative AI.

REFERENCES

[1] T. B. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems* (NeurIPS), 2020, vol. 33, pp. 1877-1901.
[2] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 246, pp. 1-113, 2023.
[3] H. Touvron et al., "LLaMA: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.
[4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637-646, Oct. 2016.
[5] Y. Tian, Z. Zhang, Y. Yang, Z. Chen, Z. Yang, R. Jin, T. Q. S. Quek, and K.-K. Wong, "An edge-cloud collaboration framework for generative AI service provision with synergetic big cloud model and small edge models," *IEEE Network*, vol. 38, no. 5, pp. 37-46, 2024.
[6] Z. Yao, Z. Tang, J. Lou, P. Shen, and W. Jia, "Velo: A vector database-assisted cloud-edge collaborative LLM QoS optimization framework," in *2024 IEEE International Conference on Web Services (ICWS)*, 2024, pp. 865-876.

TABLE III
SUMMARY OF OPEN CHALLENGES AND POTENTIAL RESEARCH DIRECTIONS IN CLOUD-EDGE COLLABORATIVE LLMS.

Open Challenge Category	**Specific Challenges**	**Potential Research Directions / Solutions (with links to studied papers where applicable)**
System Scalability & Robustness	• Managing millions of heterogeneous edge devices • Ensuring fault tolerance and graceful degradation • Network variability	• Decentralized coordination protocols (e.g., MARL in VELO [6]) • Adaptive resource management (e.g., PerLLM [16], EdgeMoE's expert management [34]) • Proactive fault detection and recovery (e.g., AdaptLink [45]) • Edge-native service mesh
Adaptive & Dynamic Scheduling	• Optimal real-time task partitioning under dynamic workloads • Balancing conflicting objectives (latency, cost, accuracy)	• Advanced Reinforcement Learning (RL) based schedulers (e.g., VELO [6]) • Predictive modeling of workload and network • Multi-objective optimization (e.g., PerLLM [16], `008ResourceAllocation...` [11])
Privacy-Preserving Collaboration	• Protecting sensitive user data during collaboration • Preventing model inversion or membership inference attacks	• Federated Learning (FL) enhancements • Secure Multi-Party Computation (SMPC) • Differential Privacy (DP) in model updates & queries • Trusted Execution Environments (TEEs) • Sketch-based/Logit-based sharing (CoGenesis [30]) • Proxy models & obfuscation (Pan et al. [46])
Energy Efficiency (Green AI)	• Minimizing energy consumption on edge devices • Reducing communication energy overhead	• Energy-aware task scheduling (e.g., PerLLM [16], `008ResourceAllocation...` [11]) • Hardware-software co-design for low-power inference • Efficient model compression and quantization (e.g., EdgeMoE [34]) • Opportunistic communication
Multimodal Integration	• Seamless fusion of diverse data types (text, image, audio, sensor) • Synchronizing multimodal streams across edge-cloud	• Unified multimodal architectures for edge & cloud • Cross-modal attention mechanisms • Efficient multimodal feature compression and transfer
Continual Learning & Personalization	• Adapting models to new data/user preferences without catastrophic forgetting • Efficient on-device learning	• Incremental learning techniques for edge models • Parameter-efficient on-device fine-tuning (PEFT) • Selective knowledge transfer protocols for updates
Trustworthy & Secure AI	• Ensuring model fairness, explainability, and robustness • Defending against adversarial attacks in distributed settings	• Adversarial training for edge-cloud models • Explainable AI (XAI) techniques for collaborative decisions • Bias detection and mitigation in distributed data

[7] M. Xu et al., "Unleashing the power of edge-cloud generative AI in mobile networks: A survey of AIGC services," *IEEE Communications Surveys Tutorials*, vol. 26, no. 2, pp. 1127-1170, 2024.

[8] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile edge intelligence for large language models: A contemporary survey," *IEEE Communications Surveys Tutorials*, Early Access, 2024.

[9] C. Yan et al., "Hybrid SD: Edge-cloud collaborative inference for stable diffusion models," arXiv preprint arXiv:2408.06646, 2024.

[10] H. Jin and Y. Wu, "Ce-collm: Efficient and adaptive large language models through cloud-edge collaboration," arXiv preprint arXiv:2411.02829, 2024.

[11] C. Liu and J. Zhao, "Resource Allocation for Stable LLM Training in Mobile Edge Computing," in *Proc. Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput. (MobiHoc '24)*, Oct. 2024.

[12] P. Zhang et al., "TinyLlama: An open-source small language model," arXiv preprint arXiv:2401.02385, 2024.

[13] M. Javaheripi, M. B. Hadhri, and S. Bubeck, "Phi-2: The surprising power of small language models," Microsoft Research Blog, Dec. 2023. [Online].

[14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.

[15] S. Nagel et al., "Up or down? Adaptive rounding for post-training quantization," in *International Conference on Machine Learning* (ICML), 2020, pp. 7197-7206.

[16] Z. Yang et al., "PerLLM: Personalized inference scheduling with edge-cloud collaboration for diverse LLM services," arXiv preprint arXiv:2405.14636, 2024.

[17] F. Cai, D. Yuan, Z. Yang, and L. Cui, "Edge-LLM: A collaborative framework for large language model serving in edge computing," in *2024 IEEE International Conference on Web Services (ICWS)*, 2024, pp. 799-809.

[18] C. Shao, X. Hu, Y. Lin, and F. Xu, "Division-of-thoughts: Harnessing hybrid language model synergy for efficient on-device agents," in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 1822-1833.

[19] K. Zhang et al., "Fast and slow generating: An empirical study on large and small language models collaborative decoding," arXiv preprint arXiv:2406.12295, 2024.

[20] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning* (ICML), 2023, pp. 19257-19274.

[21] S. Kim et al., "Speculative decoding with big little decoder," in *Advances in Neural Information Processing Systems* (NeurIPS), 2023, vol. 36, pp. 39236-39256.

[22] B. Liao et al., "Reward-guided speculative decoding for efficient LLM reasoning," arXiv preprint arXiv:2501.19324, 2025.

[23] R. Svirschevski et al., "SpecExec: Massively parallel speculative decoding for interactive LLM inference on consumer devices," in *Advances in Neural Information Processing Systems* (NeurIPS), 2024, vol. 37.

[24] Y. Kim et al., "Guiding reasoning in small language models with LLM assistance," arXiv preprint arXiv:2504.09923, 2025.

[25] T. Sun, P. Wang, and F. Lai, "Disco: Device-server collaborative LLM-based text streaming services," arXiv preprint arXiv:2502.11417, 2025.

[26] Z. Li et al., "AdaServe: SLO-customized LLM serving with fine-grained speculative decoding," arXiv preprint arXiv:2501.12162, 2025.

[27] F. Liu et al., "Kangaroo: Lossless self-speculative decoding for accelerating LLMs via double early exiting," in *Advances in Neural Information Processing Systems* (NeurIPS), 2024.

[28] H. Xia et al., "Swift: On-the-fly self-speculative decoding for LLM inference acceleration," arXiv preprint arXiv:2410.06916, 2024.

[29] Z. Hao, H. Jiang, S. Jiang, J. Ren, and T. Cao, "Hybrid SLM and LLM for edge-cloud collaborative inference," in *Proceedings of the Workshop on Edge and Mobile Foundation Models*, 2024, pp. 36-41.

[30] H. Zhan et al., "PICE: A semantic-driven progressive inference system for LLM serving in cloud-edge networks," arXiv preprint arXiv:2501.09367, 2025.
[31] D. Xu et al., "LLMCad: Fast and scalable on-device large language model inference," arXiv preprint arXiv:2309.04255, 2023.
[32] J. Wang et al., "OPT-Tree: Speculative decoding with adaptive draft tree structure," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 188-199, 2025.
[33] Y. Ding et al., "Enhancing on-device LLM inference with historical cloud-based LLM interactions," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 597-608.
[34] R. Yi et al., "EdgeMoE: Fast On-Device Inference of MoE-based Large Language Models," arXiv preprint arXiv:2308.14352, 2023.
[35] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations* (ICLR), 2022.
[36] E. Ben Zaken, Y. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2022, pp. 200-207.
[37] Y. Yao, Z. Li, and H. Zhao, "GKT: A novel guidance-based knowledge transfer framework for efficient cloud-edge collaboration LLM deployment," arXiv preprint arXiv:2405.19635, 2024.
[38] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning* (ICML), 2019, pp. 2790-2799.
[39] Y. Ding et al., "DC-CCL: Device-cloud collaborative controlled learning for large vision models," arXiv preprint arXiv:2303.10361, 2023.
[40] T. Fan et al., "FedMKT: Federated mutual knowledge transfer for large and small language models," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 243-255.
[41] Y. Shen et al., "Large language models empowered autonomous edge AI for connected intelligence," *IEEE Communications Magazine*, 2024.
[42] A. Wang et al., "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *International Conference on Learning Representations* (ICLR), 2019.
[43] Y. Hu et al., "A cloud-edge collaborative architecture for multimodal LLMs-based advanced driver assistance systems in IoT networks," *IEEE Internet of Things Journal*, 2024.
[44] J. Chen et al., "Edge-cloud collaborative motion planning for autonomous driving with large language models," in *2024 IEEE 24th International Conference on Communication Technology (ICCT)*, 2024, pp. 185-190.
[45] X. Hu et al., "AdaptLink: A heterogeneity-aware adaptive framework for distributed MLLM inference," in *AAAI 2025 Workshop on Artificial Intelligence for Wireless Communications and Networking (AI4WCN)*, 2025.
[46] Y. Pan et al., "Cloud-edge collaborative large model services: Challenges and solutions," *IEEE Network*, 2024.